



UNIVERSIDAD DE CÓRDOBA

GRADO EN INGENIERÍA INFORMÁTICA
SISTEMAS EN TIEMPO REAL



UNIVERSIDAD DE CÓRDOBA

Autores:

Álvaro Palacios Criado p02pacra@uco.es

Profesor

José Manuel Palomares Muñoz

Córdoba, 28 de febrero de 2023

Índice

1. Características deseables en los lenguajes de STR	3
2. Ada como lenguajes para Tiempo Real	3
2.1. Introducción	3
2.2. Léxico	4
2.3. Tipos de Datos	4
2.4. Instrucciones	5
2.5. Subprogramas	7
2.6. Estructura de programas	8
3. Tratamiento de los errores de ejecución	8
4. Entorno de ejecución de la aplicación	9

Índice de figuras

1.	Código para crear registros	5
2.	Acceso dinámico	5
3.	Instrucciones tipo bloque	5
4.	Instrucciones de selección	6
5.	Instrucciones case	6
6.	Bucle finito	6
7.	Bucle condicional	6
8.	Bucle condicional final	7
9.	Procedimiento	7
10.	Funciones	8
11.	Excepciones	8

1. Características deseables en los lenguajes de STR

Para un sistema en tiempo real, vale cualquier lenguaje, aunque determinados lenguajes nos facilitan la programación de aplicaciones, algunos son diseñados para la concurrencia como ADA, Occam y Java-RT, otros la consiguen con extensiones del lenguaje como C/C++. Ensamblador lo puede conseguir pero es muy complejo.

Las características deseables de los lenguajes para STR son:

- **Seguridad:** efectividad del compilador y del sistema en ejecución para detectar automáticamente errores de programación. Lo facilitan los datos tipificados (Python no), declaración previa de todas las variables y sintaxis no ambigua (ejemplo con el incremento de los punteros, ya que puede incrementar la dirección o el contenido).
- **Legibilidad:** facilidad para entender el programa sin necesidad de documentación externa adicional, ensamblador y c no son muy legibles, sin embargo Python y Java si. Se consigue con uso de comentarios descriptivos y pocas restricciones en la nomenclatura de variables y funciones.
- **Flexibilidad:** la capacidad que tiene para describir todo tipo de funcionalidad sin necesidad de recurrir a otros lenguajes o extensiones del lenguajes (el más flexible es el ensamblador). Especialmente acceso al hardware, control de procesos y tareas, control de recursos.
- **Portabilidad:** cuestión secundaria aunque cada vez tiene más importancia. Capacidad que tiene un lenguaje para ejecutarse el mismo código en diferente a la máquina. En STR esta muy acotado a la máquina en la que se va a ejecutar (Vídeo stream para diferentes versiones de Android). En particular: problemas con tamaño de registros y problemas con la precisión en la representación numérica.
- **Simplicidad:** capacidad de un lenguaje para describir con pocas sentencias de baja complejidad las diferentes funciones que se desean que ejecute una aplicación. La simplicidad reduce la probabilidad de errores, reduce costo y aumenta la seguridad y portabilidad.
- **Modularidad:** capacidad de un lenguaje para definir subrutinas dentro de su código de manera que estas subrutinas se pueden desarrollar como módulos independientes capaces de interactuar entre sí (ensamblador no tiene modularidad apenas). Imprescindible:
 - Control estricto de los parámetros de E/S.
 - Ocultación de la información de un módulo a los demás.
 - Especificación clara y precisa tanto de la función que realizan como la interfaz.
 - Posibilidad de compilación y depuración separada de cada módulo.
- **Eficacia:** capacidad de los lenguajes para garantizar una ejecución rápida con un rendimiento garantizado que sea capaz de cumplir con las restricciones temporales. Énfasis en:
 - Tamaño del código.
 - Velocidad de ejecución.
 - Uniformidad de los tiempos de ejecución.
 - Deben permitir la medición exacta de los tiempos de ejecución de cada tarea.

Dificultad en la predicción del tiempo de ejecución: variables que varían dinámicamente su tamaño como las cadenas, y los bucles de tipo while.

2. Ada como lenguajes para Tiempo Real

2.1. Introducción

ADA es un lenguaje de programación derivado de Pascal para las aplicaciones de larga duración. Desarrollado por la iniciativa del Departamento de Defensa (DoD) para desarrollar STR en SE (Cualquier desarrollo con el DoD tiene que estar especificada en ADA). Actualmente se usa ADA95 y GNU tiene un compilador.

2.2. Léxico

Identificadores: pueden contener letras mayúsculas y minúsculas, dígitos o el guión bajo. (ADA no diferencia entre minúsculas y mayúsculas).

Palabras reservadas: no pueden ser utilizadas como identificadores.

Se puede usar el carácter separador con los números para claridad (150.000 = (150000)). Se usa el carácter '#' para indicar un número en otra base. Los comentarios comienzan por --. Todas las sentencias en ADA terminan en ";".

2.3. Tipos de Datos

En aspectos generales tenemos:

- **Tipificación fuerte:** todos los datos tienen un tipo de dato. Solo se pueden realizar operaciones entre datos del mismo tipo. No se realizan conversiones de tipo, salvo que se realicen de manera explícita. Los resultados dan siempre un resultado del tipo correcto.
- **Tipificación débil:** los datos pueden no tener tipo de dato asociado. Se realizan conversiones de tipo de manera automática entre tipos de datos compatibles. Los resultados pueden tener un tipo de dato no coincidente con las entradas. (FORTRAN).
- **Precisión de los datos numéricos:** variables con el mismo tipo pueden tener precisiones diferentes en función de la arquitectura:
 - Enteros: 8, 16, 32, 64 bits.
 - Reales: 16, 32, 64, 80, 128 bits.

Rango de trabajo de las variables y la precisión entre números determinan el número de bits.

En ADA tenemos una tipificación fuerte:

- Tipos enumerados: Boolean, Character, definido por el usuario (**type** Modo **is** (Rapido, Lento)).
- Números enteros: con signo (Integer, **type** Indice **is range** 1 .. 10). Sin signo (modulares) **type** Octeto **is mod** 256.
- Números reales (coma flotante): se especifican el número de dígitos con los que se representará el número real, pero no se indica nada sobre la precisión ni el grado de exactitud sobre el rango. (Float tiene 6 dígitos de precisión). **type** Temperatura **is digits** 5 **range** 0.0 .. 100.0; (digits indica el número mínimo de dígitos para presentar el rango).
- Números reales (coma fija):
 - Ordinarios: se especifica el grado de exactitud que ha de mantenerse durante todo el rango del tipo. (Duration: se utiliza para control de tiempo. Precisión: 1E-9) **type** Punto.Fijo **is delta** 0.1 **range** -1.5 .. 1.0;
 - Decimales: se especifica el grado de exactitud y el número de dígitos de precisión que ha de tener: **type** Decimal **is delta** 0.01 **digits** 5 (-999.99 .. +999.99)

Para definir una variable usaremos $\rightarrow$ nombreVariable* : Tipo;

Para la asignación $\rightarrow$ nombreVariable := valor;

Tipos compuestos:

- String
- Array: **type** Temperatura **is array** (Indice) **of** Temperatura; (los tipos Temperatura e Indice deben estar previamente definidos). **type** matriz **is array** (1..10, 1..10) **of** Float;
- Registros: tipo de dato compuesto que permite agrupar diferentes tipos de datos en un tipo de dato abstracto.

```
type TipoRegistro is record
    variable: Tipo;
end record;
```

Figura 1: Código para crear registros

Tipos Acceso (Punteros): designan valores de otro tipos

- **type** operacionAcceso **is access** Operacion;
- Se pueden acceder a objetos creados dinámicamente: (t: operacionAcceso := **new** Operacion;)
- Si no se indica nada, los nuevos objetos se inicializan a **null**.
- Acceso dinámico:

```
t.m := Lento;
t.all := (m => Lento, temp => 20.0);
t := new Operacion' (m => Lento, temp => 20.0)
```

Figura 2: Acceso dinámico

2.4. Instrucciones

Hay varios tipos de instrucciones:

- Simples: Asignación $\rightarrow$ variable := valor; Llamada a procedimiento $\rightarrow$ funcion(parametros);
- Compuestas: tenemos varias:
 - Bloque:

```
declare
    A: integer; --Parte de declaraciones de variables
begin
    Leer(a);
    Imprimir(a);
end;
```

Figura 3: Instrucciones tipo bloque

- Selección:

```
if a>b then
    max:=a
elsif a<b then
    max:=b;
else
    max:=0;
end if;
```

Figura 4: Instrucciones de selección

- Selección por casos: se realiza sobre una variable discreta y hay que cubrir todas las alternativas de la variable.

```
Case dia is:
    when lunes
    when martes..viernes
    when sabado/domingo
    when others
end case;
```

Figura 5: Instrucciones case

- Iteración:
 - Bucle finito:

```
for i in 1..10 loop
    leer(v(i)); --la definicion de i es implicita en funcion del rango
end loop;
```

Figura 6: Bucle finito

- Bucle condicional:

```
while a<b loop
    a:=a+1;
end loop;
```

Figura 7: Bucle condicional

- Bucle condicional final:

```
loop
  leer(d);
  exit when d=0;
end loop;
```

Figura 8: Bucle condicional final

2.5. Subprogramas

La especificación se declara en una zona declarativa:

- Procedimientos sin parámetros: **produce** nombreProc;
- Procedimiento con parámetros: **produce** proc (param1: **in out** Tipo; param2: **in** Tipo:= ValorDefecto);
- Funciones: (Solo con el parámetro **in**) **function** nombreFuncion (param1: TipoA; param2: TipoB) **return** TipoC;
- Tres modos de los parámetros:
 - **In**: no se modifica el valor al ejecutar el subprograma. Puede tomar un valor por defecto.
 - **Out**: el valor de parámetro será asignado por el subprograma.
 - **In out**: el valor del parámetro es tanto de entrada como de salida.

El cuerpo del subprograma se coloca tras la zona declarativa:

- Procedimientos:

```
procedure Incremento (param1: in out integer; param2: in integer:=1) is
begin
  param1:=param1+param2;
end Incremento;
```

Figura 9: Procedimiento

- Funciones:

```
function maximo (a,b: integer) return integer is
begin
    if a>b then return a;
    else return b;
    end if;
end maximo;
```

Figura 10: Funciones

2.6. Estructura de programas

Un programa en ADA está formado por unidades de programa. Distintas unidades de programa:

- Subprograma: funciones y procedimientos.
- Paquetes: unidad lógica para definir una colección de entidades lógicamente relacionadas.
- Otras: Tareas, Objetos protegidos, Unidades genéricas

Las unidades de programa tienen 2 partes: declaración y cuerpo. Un programa en ADA esta formado por:

- Un procedimiento principal.
- Otros subprogramas o paquetes escritos por el programador.
- Subprogramas o paquetes precompilados.

3. Tratamiento de los errores de ejecución

El tratamiento de las excepciones se limita a bloques de código: cada bloque de código deberá hacer algo diferente si se produce una determinada excepción. (Sería como un try/catch en C++)

```
Declare
    Subtype Temperatura is Integer range 0..100;
Begin
    -- Lectura del sensor de Temperatura y asignación del valor
Exception
    -- Manejador
End;
```

Figura 11: Excepciones

Si un valor se sale "fuera de rango" se lanza la excepción *Constraint_Error*. No todos los bloques pueden tener manejador.

Propagación de la excepción. Cuando se genera una excepción en un bloque que no posee manejador:

1. Solución: el compilador avisaría al programador de la ausencia del manejador → elevada complejidad en tiempo de compilación.
2. Solución: buscar los manejadores que estén activos en la cadena de procedimientos en la pila de llamada actual: *Propagación de la excepción*.

Para limitar la propagación en un punto y evitar que "suba" hasta el procedimiento principal se permite el uso de manejadores "catch-all" que capture todas las excepciones.

4. Entorno de ejecución de la aplicación

Podemos diferenciar tres tipos de entornos:

- Mecanismo de creación de un ejecutable: la plataforma de desarrollo suele ser diferente a la de explotación. Se utilizan compiladores cruzados para crear aplicaciones. El ejecutable se introduce en el sistema a explotar en modo off-line. El sistema de explotación suele estar SO-free con lo que el programa tiene que incluir todas las llamadas del S.O e incluso un pequeño Kernel. En STR solo lo necesario, reducir código.
- Ejecutable autocontenido (stand-alone executable): se genera un único fichero con el código y las funciones necesarias del sistema operativo.
 - Ventaja: rapidez, se conocen a priori todas las funciones.
 - Desventajas: aumento del tamaño, imposibilita la actualización de funciones del S.O sin parar el sistema.
- Ejecutable con enlaces dinámicos (dynamically linked executable): se genera un fichero solo con el código, y el resto de funciones y recursos se encuentran en otros ficheros, y se llaman dinámicamente y bajo demanda de memoria.
 - Ventajas: reducción en el tamaño del fichero de aplicación, actualización de funciones del S.O.
 - Desventajas: velocidad de ejecución, mayor consumo de memoria, control dinámico de dependencias.